Sanitizer Fuzzer Book

セキュリティ・キャンプ全国大会 `Y1: Code Sanitizer・Fuzzer 自作ゼミ` に着想を得て、学習用コンパイラへ instrumentation を加えながら、`AddressSanitizer`、`SanitizerCoverage`、coverage-guided fuzzing の原理を 1 冊で追うためのノートです。`compilerbook` を読み終えたあとに「生成したコードをどう観測し、どう壊し、どう自動で見つけるか」を学ぶ流れを意図しています。

> **編集注記**
> この文書は公式ゼミ資料や LLVM 文書の逐語訳ではありません。公開されている一次資料をもとに、学習用の小さな C コンパイラを改造しながら理解できるように再構成した教材です。説明の中心は原理の再実装であり、LLVM/Clang の全実装を追体験することではありません。

目次

Contents

はじめに 3
- なぜ `compilerbook` の次にこれを学ぶのか 3
- サニタイザとは何か 3
- ファザとは何か 4
- サニタイザとファザの役割分担 4
- 本書の方針 4
- 対象読者 4
- 参照する一次資料 5
- 各章の見取り図 5

計測付きコード生成の入口 5
- どこを改造するのか 6
- 何を「危険な操作」とみなすか 6
- instrumentation の最小形 6
- コード生成器は何を知っているべきか 6
- AST ではなくコード生成直前に入れる理由 7
- ランタイム呼び出しを足す影響 7
- どこまでコンパイラで、どこから runtime か 7
- `malloc` をどう扱うか 7
- 最小実装の設計 7
- アドレス計算の流れをもう一度見る 8
- レポート形式を先に考える 8
- この章の出口 8

最小 `AddressSanitizer` 8
- shadow memory の発想 9
- redzone は何のためにあるか 9
- `malloc` をどう差し替えるか 9
- `free` の役割 10

quarantine の意味 10
アクセスチェック 10
heap overflow の例 10
use-after-free の例 10
レポートは何を出すべきか 11
shadow memory の配置 11
部分アクセスとアラインメント 11
本家 `AddressSanitizer` と比べて見えること 11
この章の出口 12
runtime の責務と縮小実装の限界 12
runtime が持つべき情報 12
`malloc` を差し替える戦略 12
エラー時の停止方針 13
report の見やすさ 13
shadow 更新の一貫性 13
stack や global はなぜ難しいか 13
ライブラリ関数の問題 13
false positive と false negative 14
本家 ASan のランタイムが大きい理由 14
ここで一度、検査の意味を整理する 14
次章への橋渡し 15
`SanitizerCoverage` と最小 fuzzer 15
なぜ coverage が必要か 15
edge coverage の最小実装 15
計測点をどこへ置くか 16
runtime の bitmap 16
最小 fuzzer の外形 16
fuzz target とは何か 16
mutation の最初の形 17
corpus が育つとは何か 17
sanitizer との統合 17
再現性と最小化 17
dictionary と grammar の話 17
process model の違い 18
最小実装でも見える改善点 18
この章の出口 18
付録と次の一歩 18
章ごとの復習ポイント 18
典型的なバグ例 19
heap-buffer-overflow 19
heap-use-after-free 19
double-free 19
fuzz target の作り方 19
小さな演習課題 19
LLVM/Clang 系との対応表 20
用語小事典 20

よくあるつまずき 20
sanitizer を入れたのに何も検出しない 20
coverage が増えない 20
fuzzing すると再現しない crash が出る 20
ASan と fuzzer を一緒にすると遅すぎる 20
さらに進むなら 21
参考にするとよい OSS 21
おわりに 21

はじめに

この文書は、`Code Sanitizer` と `Fuzzer` を「既製品の魔法の箱」として使うのではなく、どのような発想で作られているのかを小さな実装で追うためのノートです。主役は `compilerbook` で作ったような学習用 C コンパイラです。そのコード生成段階に instrumentation を差し込み、最小の runtime を自分で持たせることで、`AddressSanitizer`、`SanitizerCoverage`、coverage-guided fuzzing の核だけを抜き出して理解します。

セキュリティ分野では、脆弱性を「読んで知る」だけではなく、**どうやって見つけるか** を知ることが重要です。さらにその一歩先には、**どうやって自動化するか** があります。サニタイザとファザはまさにそのための道具で、前者は壊れ方を観測しやすくし、後者は壊れる入力を効率よく探します。本書では、この二つがなぜ相性がよいのかを段階的に見ていきます。

この章の到達点

本書の到達点は次の通りです。

- sanitizer は「入力を綺麗にする処理」ではなく、計測付き実行環境だと説明できる
- `AddressSanitizer` が shadow memory と runtime の組み合わせで動くことを説明できる
- coverage-guided fuzzer が edge coverage を使って corpus を育てる理由を説明できる
- sanitizer と fuzzer を一緒に使う意味を、自前実装の観点から説明できる

なぜ `compilerbook` の次にこれを学ぶのか

`compilerbook` を読み終えると、ソースコードがトークン化され、構文木になり、アセンブリへ落ちるところまで一通り追えます。すると次に自然に気になるのは、「生成したコードをどう調べるか」「実行中の危険な挙動をどう見つけるか」です。普通のアプリケーション開発ではコンパイラを箱として使いますが、その箱を少し開けて instrumentation を差し込めるようになると、検査ツールの見え方が急に変わります。

サニタイザもファザも、外から見ると単に「便利なオプション」です。たとえば `-fsanitize=address` や `-fsanitize-coverage=trace-pc-guard` を付けてビルドすればよい。しかし、その背後ではコンパイラが IR や機械語の周辺へ検査コードを差し込み、ランタイムが shadow memory や crash report を支え、coverage 情報が corpus 選別の指標になります。本書では、その裏側を「小さな車輪の再発明」として自分で作ります。

サニタイザとは何か

ここでいう `sanitizer` は、入力値を綺麗にする処理ではありません。プログラムを **計測しながら実行する仕組み** のことです。コンパイル時に検査用コードを埋め込み、実行時にそのコードがメモリアクセスや未定義動作を監視し、危険が起きた瞬間に報告して止めます。

代表的なものとしては次があります。

- `AddressSanitizer` ヒープ/スタック/グローバル周辺の out-of-bounds や use-after-free を検出する
- `UndefinedBehaviorSanitizer` C/C++ の未定義動作に当たる操作を検出する
- `MemorySanitizer` 未初期化メモリの利用を検出する
- `ThreadSanitizer` データ競合を検出する

本書ではこのうち `AddressSanitizer` を主対象にします。理由は、学習用コンパイラへ入れるにはもっとも発想が分かりやすく、かつ fuzzer と組み合わせたときの価値が直感的だからです。

ファザとは何か

`fuzzer` は、対象プログラムへ大量の入力を自動で投げ込み、クラッシュやアサーション失敗や sanitizer の検出を起こす入力を探す仕組みです。単にランダムバイト列を投げるだけでも最初の一歩にはなりますが、実用的なものは coverage を使って「新しい実行経路を増やす入力」を優先します。

つまりファザの本質は、**入力生成** と **評価指標** の組み合わせです。どんな入力を作るか。何をもって「面白い」とみなすか。クラッシュしたかだけではなく、新しい edge を踏んだか、新しい compare を越えたか、タイムアウトしたかなどを使って corpus を育てます。

サニタイザとファザの役割分担

二つは似ているようで仕事が違います。

- sanitizer は、壊れた瞬間に「ここがおかしい」と観測する
- fuzzer は、壊れる入力を効率よく探す

この役割分担があるから、相性がよいのです。ファザだけだと silent corruption を見逃すことがあります。サニタイザだけだと、どんな入力で壊れるかを人手で見つけなければなりません。両方を組み合わせると、「壊れ方の観測」と「壊れる入力の探索」が自動化されます。

本書の方針

本書は LLVM/Clang の実装を逐次読解する本ではありません。本家の実装は非常に洗練されていますが、そのまま読むと規模が大きく、shadow memory のアドレス設計、stack/global instrumentation、最適化との相互作用、例外処理、プラットフォーム差異など、多くの論点が同時に現れます。

そこで本書では、まず次の縮小版を作ります。

1. 学習用 C コンパイラに instrumentation を差し込めるようにする
2. ヒープに限定した最小 `AddressSanitizer` を作る
3. shadow memory と redzone の意味を理解する
4. edge coverage を集める最小 runtime を作る
5. その coverage を使って corpus を育てる小さな fuzzer を作る

この縮小版で核を掴んだあとに、本家 `ASan`、`SanitizerCoverage`、`libFuzzer`、`AFL++` との対応を見ます。

対象読者

前提として、次の知識があることを想定します。

- C のポインタ、配列、`malloc`/`free`
- 学習用コンパイラの構文木、コード生成、呼び出し規約

- プロセスとアドレス空間の基礎

逆に、LLVM IR や sanitizer の実運用経験は不要です。本書の役割は、その前段にある「原理がどうつながっているか」を一本の線にすることです。

> **注意**
> 本書の縮小実装は、学習のために意図的に範囲を絞ります。ヒープ専用 `ASan`、単一スレッド、in-process fuzzing が前提です。これで本家の代わりになるわけではありませんし、性能や互換性を目標にもしません。

参照する一次資料

着想の起点はセキュリティ・キャンプ全国大会 `Y1: Code Sanitizer・Fuzzer 自作ゼミ` です。公式説明でも「既存のコンパイラを改造して sanitizer や fuzzer を実装する」ことが主題として掲げられています。この方向性は学習教材として非常に優れていて、ツールの使い方ではなく「なぜその形になるか」を追いやすいからです。

技術的な比較対象としては次を使います。

- Clang `AddressSanitizer` 文書
- Clang `SanitizerCoverage` 文書
- LLVM `libFuzzer` 文書
- `AFL++` の設計資料

本書はこれらの説明をそのまま写すのではなく、小さな再実装を通して理解しやすい順に並べ替えます。

> **編集注記**
> 公式資料の世界では sanitizer も fuzzer も、かなり多くのチューニングと周辺機構を持っています。本書はその全体像を削って、学習者が「コンパイラ改造」「runtime」「coverage」「入力生成」の 4 層を一度に見通せるようにすることを優先します。

各章の見取り図

以下の順で進みます。

1. まず、学習用コンパイラのどこに instrumentation を差し込むか整理する
2. 次に、最小 `AddressSanitizer` を作り、heap overflow と use-after-free を検出する
3. その後、runtime の責務と縮小実装の限界を整理する
4. さらに、edge coverage を集める仕組みを足す
5. 最後に、その coverage を使う最小 fuzzer を作り、sanitizer と統合する

この順序には理由があります。fuzzer を理解するには、まず「壊れたかどうかをどう観測するか」が見えていたほうがよいからです。したがって本書では sanitizer を先に置きます。クラッシュと検知の違い、runtime が必要な理由、メモリ周辺の赤帯の意味が見えてくると、coverage-guided fuzzing の価値も自然に理解できます。

計測付きコード生成の入口

最初に行うのは、既存の学習用 C コンパイラを「普通のコードを出す箱」から、「観測用コードも一緒に出す箱」へ変えることです。ここでは sanitizer も fuzzer もまだ作りません。その前提として、どの地点で instrumentation を差し込むのが自然かを整理します。

この章の到達点
この章では次を押さえます。

- instrumentation は最適化の別名ではなく、意味のある追加コード生成だということ
- AST、型、左辺値、アドレス計算の情報が揃っている地点で挿し込むと実装しやすいこと
- runtime が必要な処理と、コード生成だけで完結する処理を分けて考えること

どこを改造するのか

学習用コンパイラを改造するなら、大きく分けて 3 つの差し込み位置があります。

1. 構文木の段階で専用ノードを増やす
2. 中間表現があればその段階で計測命令を差す
3. 機械語やアセンブリ生成直前で、危険な操作の前後へ呼び出しを差す

`compilerbook` 系のように薄い構成のコンパイラでは、3 番目が最も分かりやすいです。すでに式の型、左辺値かどうか、何バイト読もうとしているか、どのアドレスを触るかが見えているので、`load` や `store` の直前でチェックを差し込みやすいからです。

何を「危険な操作」とみなすか

最小の `AddressSanitizer` を作るなら、まず監視対象を絞ります。本書ではヒープメモリアクセスだけを対象とし、次の操作を危険とみなします。

- `malloc` が返した領域の左右へはみ出す読み書き
- `free` 済み領域へのアクセス
- `free` 後に二重解放する操作

この時点で大事なのは、「C の全ての未定義動作を扱おうとしない」ことです。対象を広げすぎると、コンパイラ改造と runtime 設計の両方が一気に重くなります。まずはメモリ安全性の一部を掴むことに集中します。

instrumentation の最小形

メモリアクセス前に 1 本の関数呼び出しを差し込むのが最初の形です。たとえば 4 バイト書き込みの直前なら「このアドレスへ 4 バイト書いてよいか」を runtime に尋ねます。

```
*(int *)p = x;
```

という元コードに対して、発想としては次のような形を出したいわけです。

```
__asan_check_store(p, 4);
*(int *)p = x;
```

読み込みなら `__asan_check_load`、書き込みなら `__asan_check_store` と分けてもよいですし、種別を引数で渡して 1 本化しても構いません。教材としては分けたほうが読者に分かりやすいです。

コード生成器は何を知っているべきか

この呼び出しを差し込むには、コード生成器が少なくとも次を知っている必要があります。

- どのアドレスを実際に触るか
- 何バイト読み書きするか
- それが load か store か

- そのアドレスが左辺値評価の途中結果ではなく、実際のアクセス点であること

`compilerbook` 後半で型付けを入れているのは、まさにこういう場面で効きます。`char` なら 1 バイト、`int` なら 4 バイト、ポインタなら 8 バイトというサイズ情報が、instrumentation の入力になります。

AST ではなくコード生成直前に入れる理由

構文木の段階で `CheckLoad` や `CheckStore` のようなノードを生やす手もあります。これは拡張性がありそうに見えますが、学習用コンパイラでは責務が散りやすいです。特にアドレス計算が複雑な式だと、「まだ値を読んでいないのか」「すでにロード済みなのか」を AST 上で丁寧に追う必要があります。

その点、コード生成直前なら実アクセス点がはっきりしています。どのレジスタにアドレスが入っているか、何バイトの命令を出そうとしているかが見えるため、最初の実装として素直です。

ランタイム呼び出しを足す影響

メモリアクセスごとに関数呼び出しを挟むと、当然実行速度は落ちます。本家 `AddressSanitizer` も性能コストを持ちますし、公式文書でも通常およそ 2 倍程度の slowdown が目安として説明されています。学習用実装ではさらに遅くなるでしょう。

しかしここでは速度は主題ではありません。重要なのは、**危険な操作を意味のある観測点へ変えること** です。単にメモリが壊れるのではなく、「どのアドレスに」「何バイトで」「どの種類のアクセスをしようとしたか」が runtime へ届くようにする。この意味変換が本質です。

どこまでコンパイラで、どこから runtime か

sanitizer を実装するときに混乱しやすいのは、コンパイラと runtime の責務が混ざることです。整理すると次のようになります。

- コンパイラの責務: どの場所にチェックを差し込むか決め、必要な情報を引数に載せる
- runtime の責務: 実際にそのアクセスが安全か判定し、危険なら報告して止める

この分離は非常に重要です。shadow memory の管理、`malloc`/`free` の差し替え、redzone の poison/unpoison は runtime 側です。コンパイラがそれら全てを直接知る必要はありません。

`malloc` をどう扱うか

ヒープ監視をしたいなら、まず対象プログラムが使う `malloc` と `free` を監視下へ置く必要があります。これはコンパイラだけでは完結しません。`malloc` が返した領域の周囲へ redzone を置き、その対応する shadow memory を poison し、`free` 時には使用中領域を poison して quarantine へ回す、といった操作が必要です。

つまり sanitizer は、単なる「読み書き前の if 文」ではありません。**メモリ管理の周辺ごと runtime で支える** ことで初めて意味を持ちます。ここが入力検証ライブラリとの大きな違いです。

最小実装の設計

本書では次の縮小版を採用します。

- 対象はヒープだけ
- `malloc` は内部で少し多めに確保し、左右に redzone を置く
- shadow memory は 1:8 の粗い対応表を使う

- load/store 前のチェックは関数呼び出しで行う
- 危険を見つけたら即座にレポートして終了する

この縮小版でも、heap overflow と use-after-free の核は十分見えます。stack/global instrumentation や vectorized access はいったん捨てます。

アドレス計算の流れをもう一度見る

たとえば `a[i] = 42` のような代入では、実際の生成器は概ね次のような段階を踏みます。

1. `a` の基底アドレスを得る
2. `i` を評価する
3. 要素サイズを掛けて加算し、実アドレスを作る
4. そのアドレスへ store 命令を出す

instrumentation を入れるなら 4 の直前がよいです。3 の途中でチェックすると、まだ最終アドレスが確定していないかもしれませんし、最適化や式変形があると二重にチェックを入れやすくなります。最初は「本当に触る瞬間だけ見る」が安全です。

レポート形式を先に考える

runtime を書く前に、「何が起きたら何を出したいか」を決めておくと後が楽です。最低限ほしいのは次です。

- load か store か
- アドレス
- サイズ
- どの領域が poison されていたか
- 可能なら allocation/free の履歴

本家 ASan はスタックトレースや origin 情報も豊富ですが、学習用ならここまでで十分です。重要なのは「silent corruption を error report に変える」ことです。

> **注意**
> 学習用実装では、チェック関数の呼び出し自体がターゲットプログラムの挙動を変えることがあります。レジスタ退避、スタック使用量、関数呼び出し順序が変わるためです。したがって「instrumentation を入れても意味論が完全に同じ」とは限りません。本家実装が ABI や最適化と慎重に付き合う理由はここにあります。

この章の出口

ここまでで、「どこへ何を差し込むか」は見えました。次章では、その差し込み先の runtime として最小 `AddressSanitizer` を組み立てます。shadow memory、redzone、poison/unpoison、quarantine を順に揃え、heap overflow と use-after-free を実際に検出できるところまで進めます。

最小 `AddressSanitizer`

この章では、ヒープ専用の最小 `AddressSanitizer` を組み立てます。本家 ASan は多くのプラットフォームと多数の不正アクセス種別に対応していますが、学習の芯はそこまで広くありません。**有効な領域を許し、それ以外を poison として拒否する** という考え方を掴めば、設計の骨格はかなり見えます。

この章の到達点
この章では次を理解することを目標にします。

- shadow memory が「メモリのメタデータ置き場」であること
- redzone が heap overflow を見つけやすくする理由
- `free` 後に poison し続けることで use-after-free を検出できること

shadow memory の発想

`AddressSanitizer` の中核は shadow memory です。対象プログラムの実メモリ全体を、そのまま詳細に追跡するのではなく、**粗い対応表** を別に持ちます。たとえば 8 バイトの実メモリに対して 1 バイトの shadow を割り当て、その 8 バイトがどの程度アクセス可能かを記録します。

学習用には、次のような単純な規則で十分です。

- shadow 値 `0`: 対応する 8 バイトは全部アクセス可能
- shadow 値 `1..7`: 先頭からその値バイトだけアクセス可能
- shadow 値 `0xff` のような特別値: 全部 poison

この設計の良いところは、チェックが速いことです。アドレス `addr` に対し `shadow[addr >> 3]` を見れば、おおよその安全性が分かります。精密な境界検査をするのではなく、「ほぼ 8 バイト単位」で効率的に管理する発想です。

redzone は何のためにあるか

単に `malloc(n)` の有効範囲だけを記録しても、境界をまたいだアクセスを見落としやすくなります。そこで、割り当てた領域の前後に少し余分な領域を置き、それを poison しておきます。これが redzone です。

たとえば `malloc(24)` を受けたら、実際には次のように多めに確保します。

```
[ left redzone ][ 24-byte user region ][ right redzone ]
```

ユーザ領域だけ unpoison し、左右は poison のままにしておけば、少しはみ出したアクセスが即座に検出できます。これは heap overflow の検出に非常に効きます。

`malloc` をどう差し替えるか

最小実装では、ターゲットプログラムが呼ぶ `malloc` を自前 runtime の `asan_malloc` へ差し替えます。流れは次の通りです。

1. 実際に必要サイズより大きく生メモリを確保する
2. 先頭にヘッダを置き、ユーザサイズや元ポインタを記録する
3. 左右 redzone を poison する
4. ユーザ領域だけ unpoison する
5. ユーザへ返すアドレスを返却する

このとき、ヘッダの位置とアラインメントを丁寧に扱う必要があります。学習用なら 16 バイト整列程度に固定してしまうのが実装しやすいです。

free の役割

free は単にメモリを解放するだけではありません。sanitizer の観点では、**この領域はもう触ってはならない** と宣言する操作です。そのため最小 runtime でも、free 時にユーザ領域を poison に戻します。

```
void asan_free(void *p) {
    Header *h = ptr_to_header(p);
    poison_user_region(h);
    quarantine_push(h);
}
```

重要なのは、すぐに本物の allocator へ返さず quarantine に積むことです。もし即座に再利用すると、use-after-free が別の有効領域アクセスに見えてしまい、検出しにくくなります。一定期間 poison のまま寝かせることで、「解放済みなのに触った」ことを捕まえやすくします。

quarantine の意味

quarantine は、解放済みブロックをしばらく再利用しないための待機列です。本家 ASan でも重要な仕組みで、学習用にも小さく入れる価値があります。実装はリングバッファや単純キューで十分です。

この設計は、use-after-free の再現性を上げるためにあります。C のバグは再利用タイミングに左右されやすく、解放直後に別用途で同じアドレスが使われると現象が揺れます。quarantine で遅らせると、poison されたままの時間が長くなり、観測しやすくなります。

アクセスチェック

本書の最小チェック関数は、対象アドレスとアクセスサイズを受け取り、shadow memory を見て安全かどうかを決めます。

```
void asan_check_load(void *addr, size_t size);
void asan_check_store(void *addr, size_t size);
```

チェックの考え方は次の通りです。

1. addr に対応する shadow バイトを求める
2. そのバイトが poison 値なら即失敗
3. 部分的にしか有効でない値なら、アクセス末尾が有効範囲を越えていないか見る

たとえば shadow 値が 3 なら、その 8 バイトブロックでは先頭 3 バイトだけ有効です。4 バイトアクセスで 2 バイト目から始めると範囲を越えます。このように、粗い管理でも多くの境界バグを捕まえられます。

heap overflow の例

`char *p = malloc(16); p[16] = 'x';` というコードを考えます。有効なインデックスは `0..15` なので、`p[16]` は右 redzone への 1 バイト書き込みです。`asan_check_store(&p[16], 1)` は対応する shadow を辿り、その場所が poison されていることを見てエラーにします。

このとき重要なのは、メモリ破壊が起きてから後で検出するのではなく、**書き込み前に止める** ことです。壊れたあとに結果を推測するのではなく、壊す瞬間を観測点へ変えます。

use-after-free の例

`int *p = malloc(sizeof(int)); free(p); *p = 42;` のようなコードでは、free がユーザ領域を poison に戻しておくことで、後続の store チェックが失敗します。もし quarantine がなけ

れば、運悪く同じアドレスが別用途へ再割り当てされ、バグが見えにくくなるかもしれません。

この例から分かるのは、sanitizer は単なるアクセスチェックではなく、**領域の生涯管理** でもあるということです。いつ生成され、いつ有効で、いつ死ぬのか。その情報を runtime が持つことで初めて意味のある診断ができます。

レポートは何を出すべきか

学習用実装でも、エラー報告は丁寧にしたほうがよいです。最低限、次を出したいです。

- `heap-buffer-overflow` か `heap-use-after-free` か
- load/store の種別
- 問題アドレス
- アクセスサイズ
- 対応する block の開始位置とユーザサイズ

可能なら、どの時点で `malloc` され、どの時点で `free` されたかの簡単な ID を持たせてもよいです。完全なスタックトレースがなくても、再現入力があればかなり役に立ちます。

shadow memory の配置

学習用実装では、巨大な仮想アドレス空間に本家のような固定 shadow マッピングを持たせる必要はありません。ユーザ allocator が配るヒープ範囲だけを shadow 化し、別配列で持てば十分です。

これは本家との差として意識しておくべき点です。ASan の本番実装はアドレス変換式がもっと直接的で、スタックやグローバルや mmap 領域も含めた設計になっています。本書では「考え方を掴む」ことを優先し、対象範囲をヒープへ絞ります。

部分アクセスとアラインメント

実際のアクセスは常に 8 バイト境界に揃うわけではありません。1 バイト、2 バイト、4 バイト、8 バイトの読み書きが入り混じります。したがってチェック関数は、単に `shadow[addr >> 3] == 0` を見るだけでは足りません。開始オフセットとサイズを見て、ブロック内のどこまで触ろうとしているかを計算する必要があります。

ここで理解すべきなのは、「shadow memory は粗いが、粗いままでも十分実用的な情報を持てる」という点です。C の多くのメモリ破壊は境界近辺で起きるため、8 バイト単位の管理でもかなり捕まえられます。

> **注意**
> 本書の縮小版は heap に限定しているため、スタックバッファ overflow やグローバル変数周辺の破壊は検出できません。また、ライブラリ内部の `memcpy` のような bulk access は追加対応しない限り見逃します。本家 ASan が専用 interceptor を多数持つのはそのためです。

本家 `AddressSanitizer` と比べて見えること

この縮小版を作ると、本家 ASan の複雑さがどこから来るかも見えます。

- stack/global も対象にしたい
- `memcpy` などのライブラリ呼び出しも見たい
- プラットフォームごとに shadow 配置が違う

- レポートへスタックトレースや allocation site を載せたい
- 性能コストを下げたい

つまり難しさは「原理が難しい」より、「対象と互換性を広げると設計の責務が増える」に近いです。原理そのものは比較的素朴です。

この章の出口

ここまでで heap overflow と use-after-free を検出する最小 sanitizer は見えました。しかし、まだ runtime の周辺責務と限界を十分に整理していません。次章では allocator 置き換え、reporting、quarantine 方針、未対応ケースをまとめ、なぜ本家実装がより大きなランタイムを持つのかを整理します。

runtime の責務と縮小実装の限界

sanitizer を作るとき、コンパイラ改造に目が向きがちですが、実際には runtime のほうが長く効きます。どのアドレス範囲を shadow 化するか、どうやって `malloc`/`free` を差し替えるか、エラー時に何を出すか、再利用をどう遅らせるか。これらを決めるのは runtime です。この章では、その責務を整理しつつ、本書の縮小実装がどこを切り捨てているかを明示します。

この章の到達点

この章で理解したいのは次の点です。

- sanitizer は compiler pass 単体では成立せず、runtime とセットで考える必要があること
- heap-only の縮小実装には明確な検出限界があること
- 本家 ASan が大きなランタイムを持つ理由が、機能追加と互換性対応にあること

runtime が持つべき情報

最小実装でも、runtime は次のような情報を持ちます。

- 各 allocation block の実サイズとユーザサイズ
- ユーザポインタと実際に確保した先頭ポインタの対応
- redzone の幅
- quarantine に積まれている解放済み block
- shadow memory 本体

この情報があるから、アクセスチェックの結果をただの真偽値ではなく、人間が読める診断へ変えられます。「poison されていた」で終わるのではなく、「この 24 バイト確保の右 redzone に 4 バイト書こうとした」と言えるようになります。

`malloc` を差し替える戦略

学習用実装では、対象言語のソース中に現れる `malloc`/`free` 呼び出しを専用ランタイム関数へ置き換えるのが分かりやすいです。リンク時に同名シンボルを差し込む方法もありますが、学習用コンパイラならコード生成時に明示的に `asan_malloc` と `asan_free` を呼ぶほうが見通しがよいです。

この選択は教材上かなり重要です。なぜなら、「既存コードへ何を追加したか」が目に見えるからです。サニタイザは魔法のビットではなく、メモリ管理 API ごと置き換える仕組みだと実感できます。

エラー時の停止方針

学習用実装では、危険なアクセスを見つけたら即時終了でよいです。これは単純化でもありますが、本質でもあります。メモリ破壊が起きたあとでプログラムを継続しても、その後の挙動は不安定になりやすいからです。

一方、本家の運用ではログ収集や複数エラー観測の事情もあります。ここは「学習用だから雑でよい」のではなく、「最初の目的が crash を観測可能にすることだから即時停止で十分」と理解するのが大切です。

report の見やすさ

report は派手である必要はありませんが、再現に効く情報を落とさないことが重要です。具体的には次です。

- バグ種別
- アドレス
- アクセスサイズ
- allocation size
- その block が allocated か quarantined か

もし余力があれば、簡単な allocation ID と free ID を持たせて、「何番目の `malloc` が返した領域で、何番目の `free` で死んだか」を出すと教材としてかなり良くなります。スタックトレースがなくても、入力と照合しやすくなるからです。

shadow 更新の一貫性

`malloc`、`free`、recycle のたびに shadow を正しく更新しないと、サニタイザはすぐ嘘をつきます。false positive も false negative も起こりえます。したがって runtime は「どのイベントで poison/unpoison するか」を一貫したルールで持つ必要があります。

典型的には次のようになります。

- 確保直後: redzone を poison、ユーザ領域を unpoison
- 解放時: ユーザ領域を poison
- quarantine から再利用するとき: 新しい block ルールで再初期化

この整合性は、ファイルシステムのメタデータや GC のマーク状態に少し似ています。**本体データとは別に、状態を表すメタデータを同期して更新する** のが本質です。

stack や global はなぜ難しいか

ヒープだけなら allocator を押さえればよいですが、stack や global は事情が違います。stack では関数ごとにローカル変数のレイアウトが変わり、prologue/epilogue に poison/unpoison を入れる必要があります。さらに `use-after-return` のような現象を捉えたいなら、単に関数終了時に unpoison するだけでは足りません。

global では、静的領域の配置情報と初期化順序を扱う必要があります。教材としてはかなり面白いですが、最初から入れると runtime とコンパイラの両方が一気に重くなります。そこで本書では heap-only に止めます。

ライブラリ関数の問題

C の危険なアクセスは、必ずしも自前コードの load/store 命令だけで起こるわけではありません。`memcpy`、`strcpy`、`memset` のようなライブラリ関数を通って起きることも多いです。こ

れらを本気で見たいなら、関数を interceptor で差し替えたり、呼び出し前に追加チェックを入れたりする必要があります。

学習用実装では、この層を切るのが現実的です。その代わり付録で、「本家 ASan が libc interceptor を多数持つのはなぜか」を説明すると整理しやすくなります。

false positive と false negative

どんな sanitizer にも検出限界があります。縮小実装ならなおさらです。

- false negative: stack overflow や bulk access を見逃す
- false positive: shadow 更新ミスやサイズ計算ミスで正常アクセスを誤検出する

教材としては、この両方を意識させることが大事です。ツールが「検出した」から完全に正しいわけでも、検出しなかったから安全なわけでもありません。観測装置としての限界を常に持っています。

本家 ASan のランタイムが大きい理由

学習用縮小版を作ると、本家 runtime の大きさが自然に理解できます。単に歴史的経緯で大きいのではなく、次のような責務が積み上がるからです。

- stack/global の毒化
- 標準ライブラリや allocator の interceptor
- スタックトレース収集
- さまざまな OS/ABI への対応
- report の整形と suppressions
- オプション切り替えと互換性

つまり「原理は小さいが、製品としての責務が大きい」タイプのソフトウェアです。この構造は、コンパイラ本体や OS にも共通しています。

> **編集注記**
> LLVM 系の sanitizer は compiler-rt と密接に結び付いています。本書では runtime の責務を概念単位で整理し、ソースツリーの細部よりも「なぜ pass だけでは足りないか」を掴むことを優先します。

ここで一度、検査の意味を整理する

最小 `AddressSanitizer` を作ると、メモリバグ検出は次の 4 層へ分解できます。

1. コンパイラが危険な操作の前に観測点を差し込む
2. runtime が allocation の生涯と redzone を管理する
3. shadow memory がアクセス可能性を表現する
4. report が silent corruption を意味のある失敗に変える

この分解が見えると、後半の `SanitizerCoverage` と fuzzer も同じ見方で追えるようになります。つまり「コンパイラが観測点を埋め込み、runtime が集計し、その結果を探索に使う」という形です。

> **注意**
> sanitizer の導入でプログラムが「壊れなくなる」わけではありません。壊れ方が見えや

すくなるだけです。したがって、検出できたことは嬉しい一方で、検出されていない領域への過信は禁物です。

次章への橋渡し

ここまでで、壊れた瞬間を観測する仕組みはできました。しかし、どの入力で壊れるかはまだ人手で探しています。次章では coverage instrumentation を入れ、どの入力が新しい経路を踏んだかを記録し、その情報を使って corpus を育てる最小 fuzzer を作ります。

`SanitizerCoverage` と最小 fuzzer

sanitizer が「壊れた瞬間を観測する装置」なら、fuzzer は「壊れる入力を探す装置」です。この章では、coverage instrumentation を加えた最小 runtime を作り、その情報を使って corpus を育てる in-process fuzzer を組み立てます。主役は `SanitizerCoverage` に相当する edge coverage で、これを面白さの指標に使います。

この章の到達点

この章の目標は次の通りです。

- coverage-guided fuzzing の「guided」が何を意味するか説明できる
- edge coverage を bitmap に落とす発想を説明できる
- sanitizer と fuzzer を統合すると silent corruption を効率よく見つけやすくなることを説明できる

なぜ coverage が必要か

乱数で適当に入力を投げるだけでも、たまにはクラッシュを引けます。しかし大半の時間は、すでに見た経路を何度も踏み直すことになります。そこで「この入力は前と違う実行経路を踏んだか」を測り、そのような入力だけを残すと探索効率が大きく上がります。これが coverage-guided fuzzing の核です。

重要なのは、coverage が正しさの指標ではないことです。coverage は **新しさの近似** です。新しい edge を踏んだ入力は、まだ見ていない深い条件分岐やバグへ近づいているかもしれない。だから残す。これだけです。

edge coverage の最小実装

教材として一番分かりやすいのは edge coverage です。各 basic block や分岐辺に ID を振り、実行時にその ID を bitmap へ反映します。本家 `SanitizerCoverage` の `trace-pc-guard` は、各計測点に guard 変数を置き、その値を callback へ渡す形で実装されます。学習用ではもっと単純に、コンパイラが連番 ID を埋め込んでも構いません。

たとえば次のような疑似呼び出しを各分岐周辺へ差し込みます。

```
__cov_hit(37);
```

runtime 側では `bitmap[37] = 1` でも最初は十分です。もう少し本家に寄せるなら、直前の edge ID と xor を取る AFL 風の表現もできますが、最初の教材では単純な集合でもよいです。

計測点をどこへ置くか

coverage は load/store とは違い、全てのメモリアクセスへ入れる必要はありません。むしろ制御フローの変化点に置くのが自然です。最小実装なら次を基準にすると分かりやすいです。

- `if` の then 入口
- `else` の入口
- `while` / `for` の本体入口
- 関数入口
- `switch` の各 case 入口

これだけでも、入力によってどの枝へ進んだかがかなり見えます。プログラム全体の全 edge を正確に取るより、「新しい分岐が踏まれたか」を取れれば学習上は十分です。

runtime の bitmap

coverage runtime は、入力 1 件ごとに bitmap をリセットし、実行中に hit を記録し、終了後に「今まで見ていないビットが立ったか」を調べます。新しいビットがあればその入力を corpus へ残し、なければ捨てます。

この流れは非常に単純ですが、fuzzer の核心です。入力生成アルゴリズムより先に、「何を残すか」の判定基準が必要なのです。探索は評価関数なしには成立しません。

最小 fuzzer の外形

in-process の最小 fuzzer は、概念的には次のループです。

1. corpus から 1 件選ぶ
2. 少し mutation する
3. coverage bitmap を初期化する
4. target function へ入力する
5. 新しい coverage なら保存する
6. sanitizer 失敗や crash なら保存して終了または継続する

```
while true:
  seed = choose(corpus)
  input = mutate(seed)
  reset_bitmap()
  run_target(input)
  if crashes: save_crash(input)
  if expands_coverage: corpus.add(input)
```

このループが分かれば、libFuzzer も AFL++ もかなり見通しが良くなります。違いは主に mutation、スケジューリング、process model、signal の扱い、coverage の取り方、周辺最適化です。

fuzz target とは何か

fuzzer は何でも勝手に実行してくれるわけではありません。対象となる関数、すなわち fuzz target が必要です。一般には「バイト列を受け取り、それをパース・解釈・実行する関数」を用意します。

教材としては、学習用コンパイラ自身をターゲットにするのが一番自然です。たとえば「入力文字列を C の一部としてパースし、構文木を作り、コード生成し、必要なら実行する」ような関数を対象にできます。これにより、本そのものが前の本とつながります。

mutation の最初の形

最初の mutation は素朴で構いません。

- 1 バイト反転
- 1 バイト挿入
- 1 バイト削除
- 既存断片の複製

これだけでも、条件分岐が浅いプログラムにはかなり効きます。重要なのは mutation の賢さよりも、coverage に基づいて **残す入力を選べる** ことです。guided でない random test との違いはここにあります。

corpus が育つとは何か

corpus は単なる保存フォルダではありません。探索の足場です。新しい coverage を生んだ入力だけを残すことで、「すでに有効だった変異の結果」を次の変異の素材として再利用します。これが探索の階段になります。

単なる乱数列からいきなり深い条件へ入るのは難しくても、少しずつ条件を突破する入力が残っていけば、次第に深い地点へ届きます。fuzzer の賢さは、未来の入力を直接予測することより、過去に有効だった入力をうまく温存することにあります。

sanitizer との統合

sanitizer を入れた target を fuzz すると、クラッシュの意味が豊かになります。単に segmentation fault したかどうかではなく、heap overflow や use-after-free が「検出された」時点で失敗として扱えます。つまり silent memory corruption をより早い失敗へ変換できます。

この統合はとても重要です。もし sanitizer がなければ、壊れた入力がたまたま即時クラッシュしない限り見逃すかもしれません。sanitizer があれば、危険なアクセスの瞬間に止められます。ファザはその失敗入力を保存し、再現入力として残します。

再現性と最小化

クラッシュ入力が見つかったら、次に欲しいのは再現性と最小化です。学習用最小版なら、まず「クラッシュ入力をそのまま保存する」で十分です。ただし教材としては、「なぜ最小化が欲しいのか」も触れておきたいです。

長い入力より短い入力のほうが、人間が原因を追いやすいからです。libFuzzer や AFL++ の周辺には input minimization の仕組みがありますが、本書では概念紹介に留め、「不要なバイトを削っても同じ crash が起きるなら残す」という発想だけを説明します。

dictionary と grammar の話

coverage-guided fuzzing はバイト列 mutation だけでも強いですが、入力形式が構造化されるほど限界が見えます。たとえば C ソース片や JSON のような文法を持つ入力では、単純 mutation だけではすぐ構文エラーへ落ちがちです。そこで dictionary や grammar-aware fuzzing の考え方が出てきます。

本書ではそこまで実装しません。ただし付録で、「トークン辞書を持つだけでも分岐の深い場所へ入りやすくなる」ことは触れます。学習用コンパイラを fuzz target にする場合、この話は特に自然です。

process model の違い

本書の最小 fuzzer は in-process です。これは高速で実装も簡単ですが、target が状態を汚しやすいという問題があります。プロセスを毎回分ける fork-server 型や外部プロセス実行型には、隔離の利点があります。

ここで本家との差分を整理しておくとよいです。

- `libFuzzer` in-process を前提にし、高速な反復を重視する
- `AFL++` fork-server や binary instrumentation を含む広い実践系
- `Centipede` coverage 以外の特徴量も扱う現代的な探索器

学習用としては in-process で十分ですが、この違いを知っておくと現実のツール選択が分かりやすくなります。

最小実装でも見える改善点

単純な fuzzer を作っただけでも、次の改善案が自然に浮かびます。

- どの seed を優先して変異するか
- どの mutation が新しい coverage を出しやすいか
- タイムアウトや OOM をどう扱うか
- crash の重複判定をどうするか
- dictionary をどう導入するか

この「改善点が自分で見える」ことが、再実装型教材の価値です。既製ツールを使うだけだと、設定項目が多いだけに見えますが、自前版を持つとその理由が腑に落ちます。

> **注意**
> sanitizer を有効にした in-process fuzzing では、target の状態汚染や global state の残り方に注意が必要です。入力 1 件ごとに初期状態へ戻せない target は、coverage や crash の再現性を損ないやすくなります。

この章の出口

これで、本書の主線である `ASan → coverage → fuzzer` が一通りつながりました。最後の章では、ここまでを踏まえた実践上の見方、比較対象の OSS、典型バグ、演習、参考資料をまとめます。

付録と次の一歩

本編では、学習用コンパイラへ instrumentation を入れ、heap 専用 `AddressSanitizer` と最小 coverage-guided fuzzer を作る流れを見ました。この章では、ここまでの理解を定着させるための補助線として、典型バグ、演習、用語、実用ツールとの対応関係をまとめます。

章ごとの復習ポイント

ここまでの話を短く戻すと、各章の中心は次のように整理できます。

- 導入: sanitizer は観測装置、fuzzer は探索装置であり、組み合わせると壊れ方と壊れる入力の両方を自動化できる
- instrumentation: 学習用コンパイラの load/store や分岐点の直前で計測点を差し込む
- 最小 `ASan`: shadow memory、redzone、poison/unpoison、quarantine で heap bug を検出する
- runtime: compiler pass だけでは成立せず、allocator 差し替えや report が重要になる

- coverage と fuzzer: 新しい経路を踏む入力だけを残し、corpus を段階的に育てる

この整理を持っていれば、他の sanitizer や他の fuzzer を学ぶときも、どの層が違うのかを比較しやすくなります。

典型的なバグ例

教材として試しやすい最小バグを 3 つ挙げます。

heap-buffer-overflow

```
char *p = malloc(8);
p[8] = 'X';
```

右 redzone への 1 バイト書き込みです。最小 `ASan` でも最も分かりやすく検出できます。

heap-use-after-free

```
int *p = malloc(sizeof(int));
free(p);
*p = 42;
```

quarantine を入れておくと再現性が上がりやすいです。`free` 後に poison のまま寝かせる意味がここで見えます。

double-free

```
void *p = malloc(16);
free(p);
free(p);
```

本書の最小実装でも、allocation ヘッダの状態を見れば防げます。use-after-free と違ってメモリアクセス前ではなく、allocator API そのものの不正です。

fuzz target の作り方

学習用 target としては、次の条件を満たすものが扱いやすいです。

- `const uint8_t *data, size_t size` のような形で入力を受けられる
- 入力 1 件ごとに初期状態へ戻せる
- 入力が少し変わると分岐も少し変わる
- 例外時や sanitizer 検出時に明確な失敗へ落ちる

コンパイラ系教材との接続を重視するなら、字句解析器や式パーサを target にするのがよいです。入力が文字列なので fuzz しやすく、深い分岐もあり、境界条件も多いからです。

小さな演習課題

次の順で課題を足すと理解が深まりやすいです。

1. heap-only `ASan` に double-free 検出を追加する
2. report へ allocation ID と free ID を載せる
3. coverage の記録を block 単位から edge 単位へ変える
4. corpus 選択に「最近新しい coverage を出した seed を優先する」を導入する
5. 字句解析器用の token dictionary を足す

この順序は、前ほど runtime の理解を深める課題で、後ろほど fuzzer の探索効率を上げる課題です。

この章の到達点
演習に入る前に、少なくとも次が説明できる状態にしたいです。

- `malloc` と `free` の周辺に runtime が必要な理由
- shadow memory と redzone の関係
- coverage が corpus の足切りに使われる理由
- sanitizer と fuzzer を分けて考える意味

LLVM/Clang 系との対応表

ここまでの縮小実装を、実際の OSS と対応づけると次のようになります。

- 本書の `asan_check_load/store` Clang/LLVM の instrumentation pass が差し込むチェックや compiler-rt の runtime に対応する
- 本書の shadow memory ASan の shadow mapping の縮小版に相当する
- 本書の `__cov_hit` `SanitizerCoverage` の `trace-pc-guard` や類似の callback に相当する
- 本書の最小 fuzzer loop libFuzzer の基本ループや AFL 系の corpus 育成の核心に相当する

ここで大切なのは「完全一致」ではなく、「役割が同じ」ことです。実用ツールはこの上に最適化、互換性、並列化、再現性向上の仕組みが積み上がっています。

用語小事典

- `instrumentation` プログラムへ観測用コードを埋め込むこと
- `shadow memory` 実メモリに対応するメタ情報を持つ別メモリ領域
- `redzone` 有効領域の周囲へ置く poison 済みの緩衝帯
- `poison` アクセス禁止として印を付けること
- `quarantine` 解放済みブロックをすぐ再利用せず寝かせる待機領域
- `coverage` どのコード経路を通ったかの記録
- `corpus` 将来の mutation の素材として残しておく入力集合
- `in-process fuzzing` 同一プロセス内で target を何度も呼ぶ方式
- `interceptor` 標準関数や allocator を差し替えて監視する仕組み

よくあるつまずき

sanitizer を入れたのに何も検出しない

最初に疑うべきは、load/store の差し込み位置と `malloc`/`free` の差し替えです。shadow だけあっても、その更新が allocator 連携で壊れていれば意味がありません。

coverage が増えない

計測点が少なすぎるか、bitmap の更新が入力ごとにリセットされていない可能性があります。また、target が毎回すぐ同じ構文エラーで終わっていることもあります。

fuzzing すると再現しない crash が出る

in-process 実行で状態が残っている可能性があります。global state、静的バッファ、乱数 seed、allocator 状態の残り方を疑います。

ASan と fuzzer を一緒にすると遅すぎる

正常です。両方とも実行時コストを払って観測力を得る道具だからです。学習段階では性能より観測を優先します。

さらに進むなら

本書の次に進む方向は大きく 3 つあります。

第一は sanitizer 側を深める道です。stack/global instrumentation、`UBSan`、`MSan`、interceptor 設計、stack trace 収集へ進めます。

第二は fuzzer 側を深める道です。dictionary、grammar-aware fuzzing、fork server、coverage 以外の特徴量、分散 fuzzing へ進めます。

第三は解析対象を広げる道です。ファイルフォーマット、ネットワークプロトコル、コンパイラやパーサ、JIT、VM など、target の構造を利用した fuzzing へ進めます。

参考にするとよい OSS

- Clang `AddressSanitizer` documentation `https://clang.llvm.org/docs/AddressSanitizer.html`
- Clang `SanitizerCoverage` documentation `https://clang.llvm.org/docs/SanitizerCoverage.html`
- LLVM `libFuzzer` documentation `https://llvm.org/docs/LibFuzzer.html`
- `AFL++` `https://github.com/AFLplusplus/AFLplusplus`
- セキュリティ・キャンプ 2025 `Y1 Code Sanitizer・Fuzzer` 自作ゼミ `https://www.ipa.go.jp/jinzai/security-camp/2025/camp/zenkoku/program/y.html`

> **編集注記**
> 一次資料を読むと、ここで省略した論点が多数出てきます。特に ASan の stack/global 対応、SanitizerCoverage の mode 違い、libFuzzer の mutation policy、AFL++ の process model は、本書の縮小版よりずっと広いです。縮小版の役割は、それらを読むための地図を渡すことにあります。

おわりに

サニタイザやファザは、使うだけならコンパイラオプションや CLI の話に見えます。しかし一度小さく再実装すると、それらが「コンパイラ」「runtime」「メタデータ」「探索」の 4 層でできていることがはっきり見えます。この見え方は、単にバグを見つけるためだけではなく、ツールの限界や誤検知、性能コストを理解するうえでも非常に重要です。

`compilerbook` が「コードを生成する側」の視点を与えてくれたなら、本書は「生成したコードを観測し、壊し、調べる側」の視点を補います。この二つがつながると、コンパイラオプション一つの裏側にもかなり厚い設計があることが自然に見えてきます。